Write SOQL Queries

Learning Objectives

After completing this unit, you'll be able to:

- Write SOQL queries in Apex.
- Execute SOQL queries by using the Query Editor in the Developer Console.
- Execute SOQL queries embedded in Apex by using Anonymous Apex.
- Query related records.

Get Started with SOQL

To read a record from Salesforce, you must write a query. Salesforce provides the Salesforce Object Query Language, or SOQL in short, that you can use to read saved records. SOQL is similar to the standard SQL language but is customized for the Salesforce Platform.

Because Apex has direct access to Salesforce records that are stored in the database, you can embed SOQL queries in your Apex code and get results in a straightforward fashion. When SOQL is embedded in Apex, it is referred to as **inline SOQL.**

To include SOQL queries within your Apex code, wrap the SOQL statement within square brackets and assign the return value to an array of sObjects. For example, the following retrieves all account records with two fields, Name and Phone, and returns an array of Account sObjects.

```
1 | Account[] accts = [SELECT Name,Phone FROM Account];
```

Prerequisites

Some queries in this unit expect the org to have accounts and contacts. Before yc the queries, create some sample data.

```
// Add account and related contact
Account acct = new Account(
    Name='SFDC Computing',
    Phone='(415)555-1212',
    NumberOfEmployees=50,
    BillingCity='San Francisco');
insert acct;
// Once the account is inserted, the sObject will be
// populated with an ID.
// Get this ID.
ID acctID = acct.ID;
// Add a contact to this account.
Contact con = new Contact(
    FirstName='Carol',
    LastName='Ruiz',
    Phone='(415)555-1212',
    Department='Wingo',
    AccountId=acctID);
insert con;
// Add account with no contact
```

Show More

The Name field of the Contact sObject (https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/sforce_api_objects_contact.l is a compound field (https://developer.salesforce.com/docs/atlas.en-us.object_reference.meta/object_reference/compound_fields.htm). It's a concatenation of the FirstName, LastName, MiddleName, and Suffix fields of Contact. Object Manager in Setup lists only the compound Name field. But our sample Apex code in this unit references the individual fields that make up the compound field.

Use the Query Editor

Let's try running the following SOQL example:

1. In the Developer Console, click the *Query Editor* tab.
2. Copy and paste the following into the first box under Query Editor, and then click **Execute**.

```
SELECT Name,Phone FROM Account
```

All account records in your org appear in the Query Results section as rows with fields.

Basic SOQL Syntax

This is the syntax of a basic SOQL query:

```
SELECT fields FROM ObjectName [WHERE Condition]
```

The WHERE clause is optional. Let's start with a very simple query. For example, the following query retrieves accounts and gets Name and Phone fields for each account.

```
SELECT Name,Phone FROM Account
```

The query has two parts:

1. `SELECT Name,Phone` copy: This part lists the fields that you would like to retrieve. The fields are specified after the SELECT keyword in a comma-delimited list. Or you can specify only one field, in which case no comma is necessary (e.g. `SELECT Phone` copy).
2. `FROM Account` copy: This part specifies the standard or custom object that you want to retrieve. In this example, it's Account. For a custom object called Invoice_Statement, it is Invoice_Statement__c.

Beyond the Basics

Unlike other SQL languages, you can't specify * to retrieve all fields on an object. Instead, use the `FIELDS()` copy keyword to retrieve all standard fields (`FIELDS(STANDARD)` copy), all custom fields (`FIELDS(CUSTOM)` copy), or all (`FIELDS(ALL)` copy) fields on an object. For more information, see SELECT

with the `FIELDS()` `copy` keyword), you'll get an error because the field hasn't been retrieved.

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

You don't need to specify the `Id` `copy` field in the query as it is always returned in Apex queries, whether it is specified in the query or not. For example: `SELECT Id,Phone FROM Account` `copy` and `SELECT Phone FROM Account` `copy` are equivalent statements. The only time you may want to specify the `Id` `copy` field is when it's the only field you're retrieving because you have to list at least one field: `SELECT Id FROM Account` `copy`. You may want to specify the `Id` `copy` field also when running a query in the Query Editor as the `Id` `copy` field won't be displayed unless specified.

Write SOQL Queries

Filter Query Results with Conditions

If you have more than one account in the org, they will all be returned. If you want to limit the accounts returned to accounts that fulfill a certain condition, you can add this condition inside the WHERE clause. The following example retrieves only the accounts whose names are SFDC Computing. Note that comparisons on strings are case-insensitive.

```
SELECT Name,Phone FROM Account WHERE Name='SFDC Computing'
```

The WHERE clause can contain multiple conditions that are grouped by using logical operators (AND, OR) and parentheses. For example, this query returns all accounts whose name is SFDC Computing that have more than 25 employees:

```
SELECT Name,Phone FROM Account WHERE (Name='SFDC Computing' AND NumberOfEmployees>25)
```

This is another example with a more complex condition. This query returns all of these records:

- All SFDC Computing accounts
- All accounts with more than 25 employees whose billing city is Los Angeles

```
SELECT Name,Phone FROM Account
   WHERE (Name='SFDC Computing' OR (NumberOfEmployees>25 AND BillingCity='Los Angeles'))
```

wildcard character matches any or no character. The _ character in contrast can be used to match just one character.

Order Query Results

When a query executes, it returns records from Salesforce in no particular order, so you can't rely on the order of records in the returned array to be the same each time the query is run. You can however choose to sort the returned record set by adding an ORDER BY clause and specifying the field by which the record set should be sorted. This example sorts all retrieved accounts based on the Name field.

```
SELECT Name,Phone FROM Account ORDER BY Name
```

The default sort order is in alphabetical order, specified as ASC. The previous statement is equivalent to:

```
SELECT Name,Phone FROM Account ORDER BY Name ASC
```

To reverse the order, use the DESC keyword for descending order:

```
SELECT Name,Phone FROM Account ORDER BY Name DESC
```

You can sort on most fields, including numeric and text fields. You can't sort on fields like rich text and multi-select picklists.

Try these SOQL statements in the Query Editor and see how the order of the returned record changes based on the Name field.

Limit the Number of Records Returned

You can limit the number of records returned to an arbitrary number by adding the `LIMIT n` clause where n is the number of records you want returned. Limiting the result set is handy when you don't care which records are returned, but you just want to work with a subset of records. For example, this query retrieves the first account that is returned. Notice that the returned value is one account and not an array when using `LIMIT 1`.

Combine All Pieces Together

You can combine the optional clauses in one query, in the following order:

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOQL Queries

```
SELECT Name,Phone FROM Account
                  WHERE (Name = 'SFDC Computing' AND
NumberOfEmployees>25)
                  ORDER BY Name
                  LIMIT 10
```

Execute the following SOQL query in Apex by using the Execute Anonymous window in the Developer Console. Then inspect the debug statements in the debug log. One sample account should be returned.

```
Account[] accts = [SELECT Name,Phone FROM Account
                  WHERE (Name='SFDC Computing' AND
NumberOfEmployees>25)
                  ORDER BY Name
                  LIMIT 10];
System.debug(accts.size() + ' account(s) returned.');
// Write all account array info
System.debug(accts);
```

Access Variables in SOQL Queries

SOQL statements in Apex can reference Apex code variables and expressions if they are preceded by a colon (:). The use of a local variable within a SOQL statement is called a **bind**.

This example shows how to use the `targetDepartment` copy variable in the WHERE clause.

```
String targetDepartment = 'Wingo';
Contact[] techContacts = [SELECT FirstName,LastName
                          FROM Contact WHERE
Department=:targetDepartment];
```

an account. The contacts that are associated with the same account appear in a related list on the account's page. In the same way you can view related records in the Salesforce user interface, you can query related records in SOQL.

To get child records related to a parent record, add an inner query for the child records. The FROM clause of the inner query runs against the relationship name, rather than a Salesforce object name. This example contains an inner query to get all contacts that are associated with each returned account. The FROM clause specifies the Contacts relationship, which is a default relationship on Account that links accounts and contacts.


```
SELECT Name, (SELECT LastName FROM Contacts) FROM Account WHERE
Name = 'SFDC Computing'
```

This next example embeds the example SOQL query in Apex and shows how to get the child records from the SOQL result by using the Contacts relationship name on the sObject.

```
Account[] acctsWithContacts = [SELECT Name, (SELECT
FirstName,LastName FROM Contacts)
                               FROM Account
                               WHERE Name = 'SFDC Computing'];
// Get child records
Contact[] cts = acctsWithContacts[0].Contacts;
System.debug('Name of first associated contact: '
             + cts[0].FirstName + ', ' + cts[0].LastName);
```

You can traverse a relationship from a child object (contact) to a field on its parent (Account.Name) by using dot notation. For example, the following Apex snippet queries contact records whose first name is Carol and is able to retrieve the name of Carol's associated account by traversing the relationship between accounts and contacts.

```
Contact[] cts = [SELECT Account.Name FROM Contact
                 WHERE FirstName = 'Carol' AND LastName='Ruiz'];
Contact carol = cts[0];
String acctName = carol.Account.Name;
System.debug('Carol\'s account name is ' + acctName);
```

relationship on the Invoice_Statement__c custom object.

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Query Record in Batches By Using SOQL For Loops

Write SOQL Queries

With a SOQL for loop, you can include a SOQL query within a `for` `copy` loop. The results of a SOQL query can be iterated over within the loop. SOQL for loops use a different method for retrieving records–records are retrieved using efficient chunking with calls to the query and queryMore methods of the SOAP API. By using SOQL for loops, you can avoid hitting the heap size limit.

SOQL `for` `copy` loops iterate over all of the sObject records returned by a SOQL query. The syntax of a SOQL `for` `copy` loop is either:

```
for (variable : [soql_query]) {
    code_block
}
```

or

```
for (variable_list : [soql_query]) {
    code_block
}
```

Both `variable` `copy` and `variable_list` `copy` must be of the same type as the sObjects that are returned by the `soql_query` `copy`.

It is preferable to use the sObject list format of the SOQL for loop as the loop executes once for each batch of 200 sObjects. Doing so enables you to work on batches of records and perform DML operations in batch, which helps avoid reaching governor limits.

```
insert new Account[]{new Account(Name = 'for loop 1'),
                     new Account(Name = 'for loop 2'),
                     new Account(Name = 'for loop 3')};
// The sObject list format executes the for loop once per
returned batch
// of records

```

```
13      i++;
14  }
15  System.assertEquals(3, i); // The list should have contained the
    three records
16  // named 'yyy'
17  System.assertEquals(1, i); // Since a single batch can hold up
    to 200 records and,
18  // only three records should have been
    returned, the
19  // loop should have executed only once
```

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOQL Queries

Resources

- SOQL and SOSL Reference (https://developer.salesforce.com/docs/atlas.en-us.soql_sosl.meta/soql_sosl/)

GET READY

You'll be completing this unit in your own hands-on org. Click **Launch** to get started, or click the name of your org to choose a different one.

YOUR CHALLENGE

Create an Apex class that returns contacts based on incoming parameters.

For this challenge, you will need to create a class that has a method accepting two strings. The method searches for contacts that have a last name matching the first string and a mailing postal code matching the second. It gets the ID and Name of those contacts and returns them.

- The Apex class must be called **ContactSearch** and be in the public scope
- The Apex class must have a public static method called **searchForContacts**
  - The method must accept two incoming strings as parameters

Apex Basics & Database (/content/learn/modules/apex_database?trailmix_creator_id=srebello7&trailmix_slug=salesforce-platform-developer)

Write SOQL Queries

Choose hands-on org

Koushik nelluri Salesforce CRM

Created on 5/9/2026

Launch

Check Challenge to Earn 500 Points